



9. Python: Local and Global Variables

Local for the task, Global for the team.

Previous Section:

 [8. Python: Functions](#)

Contents:

- [1. What is a Variable?](#)
- [2. Local Variables](#)
- [3. Global Variables](#)
- [3. Properties of Global Variables and Local Variables](#)
 - [Global vs Local with the Same Name](#)
 - [Modifying Global Variables Inside a Function](#)
- [Summary](#)
- [References](#)

When we start learning Python, one of the first things we encounter is the idea of storing information. Programs need a way to remember values, track changes, and reuse data later. That is where variables come in.

But as our programs grow larger, another important question appears: ***“Where can a variable actually be used?”***

Sometimes a variable only exists inside a function for a short moment. Other times, it can be accessed from almost anywhere in the program. This introduces two important concepts in Python: Local Variables and Global Variables

Understanding the difference between them is extremely important because many beginner errors happen due to variable scope. We may try to use a variable that no longer exists... or accidentally overwrite another variable without realizing it.

So in this section, we will explore: What variables really are? How local variables work? Why scope matters in programming?

And later, we will see how global variables behave differently.

1. What is a Variable?

Before discussing local and global variables, let us quickly review what a variable actually is. A variable is simply a name used to store a value. That value can be almost anything: Numbers; Strings; Lists; Boolean values; Objects; And many more

Think of a variable like a labeled container. Instead of remembering the actual value directly, we give it a name so we can access it more easily later.

```
name = "Iris"  
age = 18  
height = 1.65
```

In this example: `name` stores a string; `age` stores an integer; `height` stores a floating-point number

We can then use these variables throughout the program:

```
print(name)  
print(age)  
print(height)
```

```
Iris  
18  
1.65
```

Variable names can usually be anything meaningful, but there are a few important rules:

- They must not begin with a number

- They cannot contain spaces
- They should not use Python built-in function names

For example:

```
# Valid
student_name = "Luna"
score1 = 95

# Invalid
lname = "Error"
my score = 100
print = "Hello"
```

The last example is especially dangerous because it overwrites Python's built-in `print()` function.

Variables may look simple at first, but they are one of the most fundamental building blocks in programming. Almost every program relies on them.

2. Local Variables

Local variables are variables created inside a function. They only exist while that function is running.

Once the function finishes execution, the variable disappears and cannot be accessed anymore.

This is why they are called local variables → they only belong to their local scope (the function itself).

Let us see a simple example:

```
def greet():
    message = "Hello World"
    print(message)

greet()
```

Hello World

Here: `message` is created inside the function; the variable only exists within `greet()`

But what happens if we try to use it outside?

```
def greet():  
    message = "Hello World"  
  
greet()  
print(message)
```

`NameError: name 'message' is not defined`

Why did this happen? Because `message` was a local variable. It only existed during the execution of `greet()`. Once the function ended, Python removed it from memory.

Another important catch:

```
def greet():  
    message = "Hello World"  
  
message = greet()  
print(message)
```

`None`

At first glance, this feels strange. ***"Wait... didn't the function create 'Hello World'? Why is the result None?"***

The answer is very important: The function created the variable `message`, but it never returned anything. In Python, if a function does not explicitly use the `return` statement, Python automatically returns a special value called `None`.

To actually send the value outside the function, we must use `return`.

```
def greet():  
    message = "Hello World"  
    return message  
  
message = greet()  
print(message)
```

Hello World

Now the function returns the local variable back to the main program, allowing us to store and use it outside the function.

This also reveals something very important about local variables:

- They normally stay trapped inside the function
- The `return` statement is one of the main ways to send their values outside

Overall, these behaviors are actually very useful because it helps functions stay independent from one another.

For example:

```
def student1():  
    score = 90  
    print(score)  
  
def student2():  
    score = 75  
    print(score)  
  
student1()  
student2()
```

90

75

Even though both functions use a variable named `score`, they do not conflict with each other because each variable exists only within its own function.

This idea is called variable scope. A local variable exists only inside its function, cannot be accessed outside, helps prevent accidental conflicts between parts of a program

And as programs become larger, this becomes extremely important for keeping code organized and manageable.

3. Global Variables

Global variables are variables declared outside all functions. Unlike local variables, they are not limited to a single function scope. This means they can be accessed from almost anywhere in the program, including inside functions.

Let us see a simple example:

```
message = "Welcome to Python"

def greet():
    print(message)

greet()
```

Welcome to Python

Here: `message` is created outside the function. But the function `greet()` can still access it. This makes `message` a global variable.

Global variables continue to exist throughout the entire execution of the program unless they are changed or removed.

We can also use them outside functions:

```

message = "Global Variable"

def display():
    print(message)

display()
print(message)

# Change the variable
message = "This is a variable"
display()
print(message)

```

Global Variable
Global Variable
This is a variable
This is a variable

The same variable was accessible both inside the function and outside the function. This is the key difference between local and global variables.

However, there is something important we must be careful about. A function can read a global variable normally, but modifying it is different.

For example:

```

count = 10

def update():
    count = count + 1
    print(count)

update()

```

UnboundLocalError:
local variable 'count' referenced before assignment

Why does this happen? Because the moment Python sees: `count = count + 1` inside the function, it assumes `count` is now a new local variable. But the function tries to use that local variable before it even exists.

To modify a global variable inside a function, we must explicitly tell Python to use the global version using the `global` keyword.

```
count = 10

def update():
    global count
    count = count + 1
    print(count)

update()
print(count)
```

11
11

Now Python understands that we are referring to the global variable instead of creating a new local one.

Global variables are powerful because they allow different parts of a program to share information. However, using too many of them can make programs harder to debug and manage because any function may change them unexpectedly. This is why programmers usually prefer local variables whenever possible, and only use global variables when shared access is truly necessary.

3. Properties of Global Variables and Local Variables

Now that we understand what local and global variables are, let us examine some important behaviors between them.

One of the most interesting parts is what happens when:

- A global variable and a local variable share the same name
- A function tries to modify a global variable

These situations introduce the idea of variable scope priority.

Global vs Local with the Same Name

What happens if a global variable and a local variable have exactly the same name?

In Python, the local variable takes priority inside the function. This means:

- Inside the function → Python uses the local variable
- Outside the function → Python still uses the global variable

Let us see an example:

```
a = "I am Global Iris"

def print_name():
    a = "I am Local Iris"
    print(a)

print_name()
print(a)
```

```
I am Local Iris
I am Global Iris
```

Why does this happen? Inside `print_name()`, Python sees: `a = "I am Local Iris"`. So Python creates a new local variable named `a`. That local variable temporarily hides the global variable while the function is running.

But once the function ends, the local variable disappears, and the original global variable remains unchanged. This is why: `print_name()` prints the local version, `print(a)` outside prints the global version

So in general: If a local variable and global variable share the same name, the function will prioritize the local one.

Modifying Global Variables Inside a Function

Reading a global variable inside a function is easy. But modifying it is different. Without special instructions, Python assumes variables assigned inside functions are local variables.

Example:

```
a = 1

def modify1():
    print("modify1:", a)

def modify2():
    a = 2
    print("modify2:", a)

def modify3():
    global a
    a = 4
    print("modify3:", a)

print("global:", a)
modify1()
print("global:", a)
modify2()
print("global:", a)
modify3()
print("global:", a)
```

```
global: 1
modify1: 1
global: 1
modify2: 2
global: 1
modify3: 4
global: 4
```

- The `modify1()` function returns `1` because the function is only reading the global variable.
- Now look at `modify2()`. Inside this function, Python created a completely new local variable named `a`. So the function changed only the local variable and the original global variable remained untouched. This is extremely important. Even though both variables share the same name, the local variable exists only inside the function, and the global variable still exists outside unchanged
- Now to actually modify the global variable itself. Python requires the `global` keyword, as seen in `modify3()` :: The output is `4` because `global a` tells Python: ***"Do not create a local variable. Use the global variable instead"***. So the function directly updates the global variable itself.

This also explains an important rule. In the case of a global and local variable having the same name:

- Without `global` → the function only changes the local variable
- With `global` → the function modifies the actual global variable

Summary

The following table summarizes the similarities and differences between local and global variables.

Property	Local Variable	Global Variable
Declared	Inside a function	Outside all functions
Scope	Only inside its function	Entire program
Lifetime	Exists only during function execution	Exists throughout program execution
Accessible Outside Function	No	Yes
Accessible Inside Function	Yes	Yes
Priority Inside Function	Higher priority if same name exists	Lower priority if local variable has same name
Modification Inside Function	Normal	Requires <code>global</code> keyword

Property	Local Variable	Global Variable
Common Purpose	Temporary calculations and isolated logic	Shared data across functions
Error Possibility	<code>NameError</code> if accessed outside	<code>UnboundLocalError</code> if modified incorrectly inside function

Understanding global and local variables is extremely important in Python because variable scope affects how data flows throughout a program. Many beginner mistakes happen simply because a variable exists in a different scope than expected.

However, as a general programming practice, it is usually recommended to avoid using global variables whenever possible. Why? Because global variables can be modified from many different places in a program, making the code harder to debug, harder to maintain, more difficult to understand in large projects. If too many functions depend on the same global variable, changing one part of the program may accidentally affect another part unexpectedly.

Local variables are usually safer because they only exist inside their function, they reduce unintended side effects, they help functions remain independent and reusable.

So while global variables are useful in certain situations, good programmers typically prefer using local variables, returning values with `return`, passing data through function parameters. Instead of relying heavily on global state.

In short:

- Learn both local and global variables carefully
- Understand how scope works
- But whenever possible, prefer local variables for cleaner and safer programs.

References

<https://www.geeksforgeeks.org/python/global-local-variables-python/>

<https://programming-25.mooc.fi/part-6/4-scope-of-variables>

Next Section:

</> 10. Python: Lists